JAR OF THOUGHTS

Technical Design Document

A Personal Emotional Keepsake Progressive Web Application

Version 1.0

June 2026

Author: Sahaji Perera

1. Project Overview

1.1 Purpose

Jar of Thoughts is a personal emotional keepsake Progressive Web Application (PWA). It digitally recreates the real-life practice of writing feelings, memories, and reflections on paper chits and storing them in physical jars. The application prioritises emotional connection, privacy, and a warm tactile user experience over productivity or social features.

1.2 Design Philosophy

Principle	Implementation
Privacy first	All data stored client-side in localStorage. No accounts. No server.
Warmth	Illustrated wooden shelf, glass jars, handwritten labels, paper textures
Simplicity	No complex navigation. One shelf. Open a jar. Read or write.
Offline first	Full functionality with no internet after first load
Cross-platform	Single codebase targets web browser, Android PWA, and native APK

1.3 Scope

In Scope

- Thought creation, reading, archiving, and deletion
- Time capsule locking with future unlock dates
- Jar management (add, rename, delete custom jars)
- 5 colour themes each with light and dark variants
- Responsive shelf rack layout system
- Full offline support via service worker
- Android APK packaging via Capacitor
- Data backup and restore (JSON export/import)

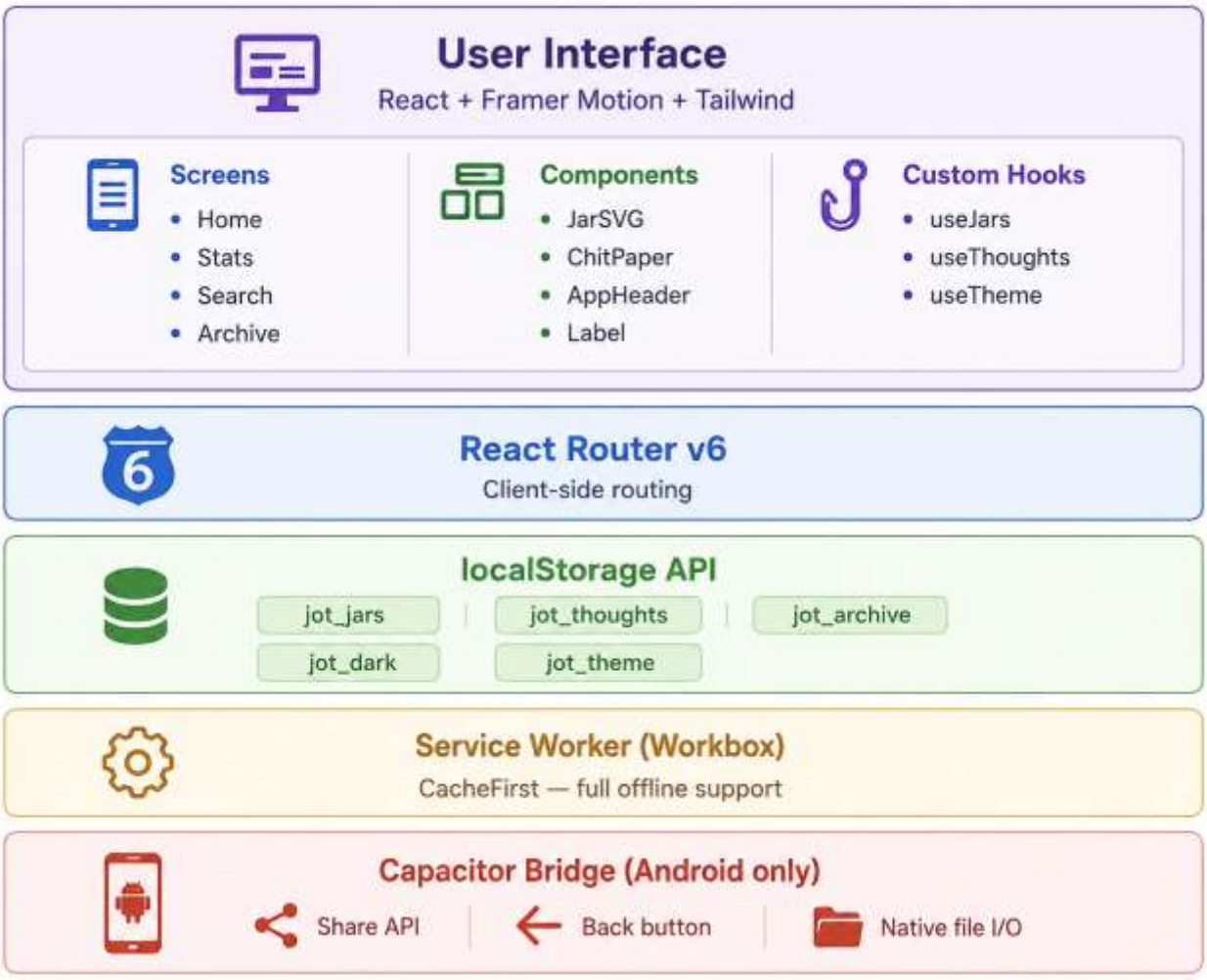
Out of Scope

- Social sharing or public profiles
- AI-generated content
- Real-time multi-device cloud sync (planned, not yet implemented)
- Push notifications

2. System Architecture

2.1 Layer Overview

The application follows a client-only architecture with no server component. All persistence is handled by the browser localStorage API. The service worker provides offline capability by caching all static assets on first load.



2.2 Key Architecture Decisions

No backend. The application is intentionally serverless. All persistence uses the browser localStorage API. This eliminates authentication complexity, server costs, and privacy concerns. The trade-off is that data is device-local; the Backup & Restore feature mitigates this.

No global state library. React built-in useState combined with custom hooks provides sufficient state management. Each hook wraps localStorage reads/writes and exposes a clean API to screens.

Single-file components. Each screen is self-contained. Shared logic is extracted into hooks and shared UI into components to avoid prop-drilling.

3. Technology Stack

Category	Technology	Version	Purpose
Framework	React	18.x	UI rendering and component model
Build Tool	Vite	8.x	Dev server, HMR, production bundling
Styling	Tailwind CSS	v4	Utility classes; inline styles for dynamic theming
Animations	Framer Motion	11.x	Drag interactions, spring animations, transitions
Routing	React Router	v6	Client-side SPA navigation
Date handling	dayjs	1.x	Relative time, date formatting
Icons	Lucide React	0.383.x	SVG icon set
PWA	vite-plugin-pwa	latest	Service worker generation, manifest injection
Service Worker	Workbox	bundled	CacheFirst strategy, offline asset caching
Android	Capacitor	6.x	Native Android bridge, Share API, back button
Fonts	Google Fonts	—	Playfair Display (serif), Inter (sans-serif)

NOTE	*100% free stack. No paid services, no subscriptions, no backend infrastructure costs.*

4. Project Structure

jar-of-thoughts/

```
├── android/ # Capacitor Android project
│   └── app/src/main/res/ # App icons, splash screens
│
├── public/
│   ├── manifest.json # PWA manifest
│   ├── icon-192.png # PWA / Android icon
│   └── icon-512.png # PWA / Android icon (large)
│
├── src/
│   ├── hooks/
│   │   ├── useJars.js # Jar CRUD — localStorage
│   │   ├── useThoughts.js # Thought CRUD + archive
│   │   └── useTheme.js # Shared theme colour reader
│   │
│   ├── components/
│   │   ├── JarSVG.jsx # Illustrated glass jar SVG
│   │   ├── ChitPaper.jsx # Draggable paper chit overlay
│   │   ├── AppHeader.jsx # Shared top navigation bar
│   │   └── Label.jsx # Handwritten jar label
│   │
│   ├── screens/
│   │   ├── Home.jsx # Shelf rack system + interactions
│   │   ├── Stats.jsx # Analytics dashboard
│   │   ├── Search.jsx # Full-text thought search
│   │   └── Archive.jsx # Archived thoughts management
│   │
│   ├── App.jsx # Router setup
│   ├── main.jsx # React DOM entry point
│   └── index.css # Tailwind import + global styles
│
└── vite.config.js # Vite + PWA plugin configuration
```

├── capacitor.config.ts # Capacitor Android configuration
├── package.json
└── README.md

5. Data Architecture

5.1 localStorage Schema

All data is stored as JSON strings in localStorage. Keys are prefixed with jot_ to avoid collisions with other applications.

5.1.1 jot_jars — Jar Array

Field	Type / Description
id	string — crypto.randomUUID()
name	string — Display name e.g. "Love"
emoji	string — Single emoji e.g. "♡"
color	string — Hex colour e.g. "#f4b8cb"

Default Jars (pre-seeded on first load)

ID	Name	Emoji	Colour
love	Love	♡	#f4b8cb
life	Life	🌿	#a8d5a2
sadness	Sadness	🌧	#9ec5e8
neutral	Neutral Thoughts	☁	#c8c8c8
dreams	Dreams	✨	#c5b8f0
gratitude	Gratitude	🌻	#f5cc7a
unsent	Unsent Words	🔥	#f0a890

5.1.2 jot_thoughts — Thought Array

Field	Type / Description
id	string — crypto.randomUUID()
text	string — Raw thought text
jarId	string — References Jar.id
createdAt	string — ISO 8601, auto-captured at save time. Never user-entered.
isLocked	boolean — True if time capsule is active
unlockAt	string | null — ISO 8601 unlock date, or null

5.1.3 jot_archive — Archive Array

Same schema as Thought. Thoughts moved here via archiveThought() are removed from jot_thoughts and appended to jot_archive.

5.1.4 Preference Keys

Key	Type	Values	Default
jot_dark	string	"true" / "false"	"false"
jot_theme	string	"amber" / "sage" / "rose" / "slate" / "lavender"	"amber"

5.2 localStorage Write Pattern

Every mutation follows the same read → mutate → write → setState pattern:

```
function addThought({ text, jarId, isLocked, unlockAt }) {
const thought = {
id: crypto.randomUUID(),
text,
jarId,
createdAt: new Date().toISOString(), // always auto-captured
isLocked,
unlockAt,
}
const updated = [...thoughts, thought]
localStorage.setItem("jot_thoughts", JSON.stringify(updated))
setThoughts(updated)
}
```

5.3 Time Capsule Logic

- When isLocked === true and unlockAt is set, the chit is visible in the jar as a sealed entry showing only the unlock date
- The chit cannot be opened or read until new Date(unlockAt) <= new Date()
- On every app load, unlockDueThoughts() scans all thoughts and auto-unlocks any whose date has passed
- Locked capsule count is shown in the quick stats row on the home screen

6. Component Specifications

6.1 JarSVG

Path: src/components/JarSVG.jsx

Prop	Type / Default / Description
color	string — required — Hex colour for liquid fill tint
count	number — required — Thought count; drives fill level
isOpen	boolean — false — Animates lid upward when true
width	number — 80 — SVG render width in px
height	number — 110 — SVG render height in px

Fill level formula:

fillLevel = Math.min(0.82, Math.max(0.05, count * 0.11))

SVG layers (bottom to top): drop shadow → jar body base → liquid fill → liquid surface wave → glass overlay → glass outline → neck → lid group (animates on isOpen) → label area.

6.2 ChitPaper

Path: src/components/ChitPaper.jsx

A full-screen overlay containing a draggable paper card. When drag starts, two circular drop zones appear at the bottom corners.

Prop	Type / Description
thought	Thought — The thought object to display
jar	Jar — The jar this thought belongs to
onDrop	function(point: {x,y}) — Called when drag ends
onClose	function — Called when backdrop is tapped
t	ThemeObject — Current theme colour tokens

Drop zone detection:

const W = window.innerWidth, H = window.innerHeight

const inFire = point.x < W * 0.45 && point.y > H * 0.6

const inArchive = point.x > W * 0.55 && point.y > H * 0.6

6.3 useTheme Hook

Path: src/hooks/useTheme.js

Reads jot_theme and jot_dark from localStorage and returns a flat colour token object. Polls every 300ms so theme changes on Home propagate to Stats/Search/Archive without a page reload.

```
const c = useTheme()
// c.bg, c.surface, c.text, c.muted, c.accent, c.border, c.card, c.lines
```

7. Screen Specifications

7.1 Home Screen — Route: /

Responsibilities: renders the full shelf rack system, manages all modal state, handles jar tap/double-tap/long-press, manages chit drag-to-delete and drag-to-archive, displays fixed bottom bar.

State	Purpose
openJarId	Which jar panel is open
writeJarId	Which jar write modal is open
viewThought	Which thought chit is showing (draggable)
burning	Fire burn overlay active
archiveOpen	Steel box lid animation
jarsPerRow	Computed from ResizeObserver

7.2 Stats Screen — Route: /stats

- Summary cards: total thoughts, jar count, time capsules, most used jar
- Date range: first thought date to latest thought date
- Donut chart: jar distribution with colour-coded segments and percentages (SVG, no library)
- 6-month bar chart: writing activity per month (CSS bars, theme accent colour)
- At-a-glance grid: active days, average words per thought, journey span, locked capsules
- Export / Import buttons for full data backup and restore

7.3 Search Screen — Route: /search

- Minimum 2 characters to trigger search
- Searches thought text case-insensitively across all jars
- Excludes locked time capsule thoughts from results
- Results grouped by jar with colour dot, emoji, and name
- Each result shows first 2 lines of thought and relative time
- Clear button appears in search input when text is entered
- Tapping a result navigates to the chit view for that thought

7.4 Archive Screen — Route: /archive

- Archived thoughts displayed in reverse chronological order
- Each card shows: jar tag, full thought text, creation date, relative time
- Restore: moves thought back to its original jar
- Delete: permanently removes from archive

8. Theme System

8.1 Overview

Five named themes, each with a light and dark variant — 10 total combinations. Theme and dark mode are stored independently in localStorage and combined at render time.

Theme	Character
Amber	Warm brown — the default; candlelit wooden shelf
Sage	Forest green — calm, nature-inspired
Rose	Dusty rose / mauve — soft, romantic
Slate	Cool blue-grey — clean, minimal
Lavender	Soft purple — dreamy, gentle

8.2 Key Theme Tokens

Token	Usage
pageBg	Full-page background gradient
shelfBg	Wooden shelf plank gradient
shelfBorder	Shelf top border colour
panelBg	Bottom sheet / modal background
panelText	Modal primary text
panelMuted	Modal secondary text
chitBg	Paper chit background
chitLines	Ruled lines on paper texture
writeBtnBg	Primary action button (accent)
labelInk	Handwritten jar label text colour
lowerShelfBg	Bottom shelf gradient (fire + archive)
ambientFire	Radial glow behind fire pit

8.3 Theme Application Pattern

```
const theme = THEMES[themeName] || THEMES.amber
const t = dark ? theme.dark : theme.light
// All inline styles reference t.tokenName
// e.g. style={{ background: t.panelBg, color: t.panelText }}
```

9. Shelf Rack Layout System

9.1 Design Requirements

- All jars — default and custom — participate in a single responsive layout
- Jars wrap into horizontal shelf rows based on screen width
- Reflows on: page load, screen resize, new jar added
- No fixed row or column counts — fully dynamic
- At least 2 empty racks always visible
- Fire pit and archive box fixed at bottom, independent of shelf scroll area

9.2 Constants

JAR_W = 72px // jar cell width
JAR_GAP = 8px // gap between jars
JAR_CELL = 80px // total cell width including gap

9.3 jarsPerRow Computation

const w = containerRef.current.clientWidth - 24 // subtract padding
const n = Math.max(2, Math.floor((w + JAR_GAP) / JAR_CELL))
setJarsPerRow(n)

9.4 Responsive Breakpoints (approximate)

Screen Width	Jars Per Row
< 360px (small phone)	2–3
360–480px (standard phone)	4
480–768px (large phone)	5–7
768–1024px (tablet)	8–10
> 1024px (desktop)	11+

9.5 Reflow Triggers

- ResizeObserver fires on page load and on every container width change
- useEffect on jars.length calls computeJarsPerRow when a new jar is added
- Framer Motion layout prop handles smooth reflow animation automatically

10. PWA & Offline Support

10.1 Manifest

Location: public/manifest.json

{

"name": "Jar of Thoughts",

"short_name": "JarOfThoughts",

"display": "standalone",

"background_color": "#F9F5EE",

"theme_color": "#B5804A",

"icons": [

{ "src": "icon-192.png", "sizes": "192x192", "type": "image/png" },

{ "src": "icon-512.png", "sizes": "512x512", "type": "image/png" }

]

}

10.2 Service Worker Strategy

Configured via vite-plugin-pwa. Strategy: CacheFirst for all static assets. Google Fonts cached separately under a named cache. Auto-updates service worker on new deployments.

10.3 Android Install Flow

1. Deploy to Vercel: npx vercel --prod
2. Open URL in Chrome on Android
3. Three-dot menu → Add to Home Screen
4. App installs with custom icon, runs full-screen, works offline

11. Android (Capacitor)

11.1 Native APIs Used

API	Usage
@capacitor/share	Export: shares JSON backup via native share sheet
Back button override	Navigates to Home instead of exiting the app

11.2 Build Process

npm run build # Vite production build → dist/

npx cap sync android # Copies dist/ into android/ project

Open android/ in Android Studio

Build > Generate Signed APK OR Run on device

11.3 Back Button Override

import { App } from '@capacitor/app'

App.addListener('backButton', () => {

if (window.location.pathname !== '/') {

navigate('/')

} else {

App.exitApp()

}

})

11.4 Export on Android

Browser Blob URL download does not work inside Capacitor WebView. Solution uses the native Share API:

```
import { Share } from '@capacitor/share'
await Share.share({
title: 'Jar of Thoughts Backup',
text: JSON.stringify(backupData),
dialogTitle: 'Save your backup',
})
```

12. Backup & Restore

12.1 Export

```
const backup = {
version: 1,
exportedAt: new Date().toISOString(),
jars: JSON.parse(localStorage.getItem('jot_jars') || '[]'),
thoughts: JSON.parse(localStorage.getItem('jot_thoughts') || '[]'),
archive: JSON.parse(localStorage.getItem('jot_archive') || '[]'),
settings: {
theme: localStorage.getItem('jot_theme') || 'amber',
dark: localStorage.getItem('jot_dark') || 'false',
}
}
```

12.2 Import

```
const data = JSON.parse(fileContent)

if (data.version !== 1) throw new Error('Incompatible backup version')

localStorage.setItem('jot_jars', JSON.stringify(data.jars))

localStorage.setItem('jot_thoughts', JSON.stringify(data.thoughts))

localStorage.setItem('jot_archive', JSON.stringify(data.archive))

localStorage.setItem('jot_theme', data.settings.theme)

localStorage.setItem('jot_dark', data.settings.dark)

window.location.reload()
```

13. Animation System

All animations use Framer Motion. No CSS @keyframes. Fire flame animations use Framer Motion animate prop on SVG paths.

Interaction	Type	Config	Notes
Jar cards on mount	Fade + slide up	Stagger 60ms, spring	Entrance animation
Jar lift on open	translateY -8px	Spring stiffness 280	Plus drop shadow boost
Jar lid animation	translateY + rotate	CSS transition inside SVG	Lid lifts when open

Panel slide up	y: 100% → 0	Spring stiffness 260	Bottom sheet
Panel drag close	Drag y, threshold 60px	dragElastic: 0.2	Swipe to close
Chit paper appear	Scale 0.85 → 1 + fade	Spring stiffness 300	Paper card
Chit drag	Free drag	dragMomentum: false	Drag to fire/archive
Drop zones appear	Scale + fade in	On drag start	Fire + archive circles
Burn overlay	Opacity pulse + shrink	Duration 1.2s	Full screen fire
Fire flames	ScaleY loop, 3 paths	Infinite stagger	Animated SVG
Steel box lid	RotateX -70deg	Spring on isOpen prop	Archive box opening
Jar reflow	Layout animation	Framer layout prop	On resize / add jar

14. Known Limitations

Issue	Cause	Status
Data is device-local	localStorage by design	Mitigated by Backup & Restore
Rename/delete requires page reload	Hook does not watch external mutations	Acceptable — reload is instant
localStorage size limit	~5–10MB per browser origin	Unlikely to be hit (text only)
Dark mode label visibility	Light ink colour in dark theme	Fix pending
Time capsule push notifications	No backend / FCM configured	Out of scope for MVP
Blob download in Capacitor	WebView blocks blob URLs	Fixed via Share API

15. Planned Features

15.1 Firestore Cloud Sync (Paused)

STATUS	*Firebase project created. Firestore + Auth enabled. Login screen and Firestore hooks written. Not yet integrated into production.*

Planned flow:

5. Optional sign-in screen (email + password)
6. On sign-in: upload local data to Firestore under users/{uid}/jars and users/{uid}/thoughts
7. On subsequent loads: merge remote → local
8. Real-time Firestore listener for multi-device sync

15.2 Other Planned Items

- Push notifications for time capsule unlocks (requires FCM)
- Jar sorting and reordering by drag
- Thought search within a single jar
- Word count display while writing
- Additional theme options

16. Dependency Reference

Production Dependencies

Package	Purpose
react / react-dom	UI framework
react-router-dom	Client-side routing
framer-motion	Animations and drag interactions
dayjs	Date formatting and relative time
lucide-react	SVG icon set
@capacitor/core	Capacitor core bridge
@capacitor/android	Android platform
@capacitor/app	Back button / app lifecycle
@capacitor/share	Native share sheet for export

Development Dependencies

Package	Purpose
vite	Build tool and dev server
@vitejs/plugin-react	React JSX transform for Vite
tailwindcss	Utility CSS framework (v4)
@tailwindcss/vite	Tailwind v4 Vite integration
vite-plugin-pwa	PWA manifest + service worker generation